MOBILE APPLICATION DEVELOPMENT

Kigali City Services & Places Directory

Individual Assignment 2 — Submission Document

Student	Jongkuch Anyar
Email	j.anyar@alustudent.com
Course	Mobile Application Development
Assignment	Individual Assignment 2
Platform	Flutter (Android & iOS)
Backend	Firebase Authentication & Cloud Firestore
State Management	Provider
Submission Date	March 2026

Submitted in partial fulfillment of the course requirements

Table of Contents

1. Implementation Reflection 3
1.1 Firebase Authentication Integration 3
1.2 Cloud Firestore Integration 4
1.3 Google Maps Integration 5
1.4 State Management with Provider 6
2. GitHub Repository 7
3. Design Summary 8
3.1 Firestore Database Structure 8
3.2 Listing Data Model 9
3.3 State Management Architecture 10
3.4 Design Trade-offs & Technical Challenges 11

1. Implementation Reflection

This section documents the key integration challenges encountered while building the Kigali City Services & Places Directory application with Firebase Authentication, Cloud Firestore, and the Google Maps Flutter SDK. For each challenge, the specific error encountered is described along with the steps taken to resolve it.

1.1 Firebase Authentication Integration

Integrating Firebase Authentication was the first step in the project. The initial setup required configuring the Firebase project through the FlutterFire CLI and enabling Email/Password authentication in the Firebase Console.

Error 1 — Missing `google-services.json` / Firebase Not Initialized

Runtime Error on App Launch

```
[core/no-app] No Firebase App '[DEFAULT]' has been created - call Fireba
se.initializeApp()
```

The app crashed immediately on launch before any UI was rendered. This occurred because `Firebase.initializeApp()` had not been called before `runApp()`, and the platform-specific configuration files (`google-services.json` for Android, `GoogleService-Info.plist` for iOS) were missing from the project.

Resolution

The FlutterFire CLI was used to automatically generate the configuration files and the `firebase_options.dart` file:

```
dart pub global activate flutterfire_cli
flutterfire configure
```

`main()` was updated to call `Firebase.initializeApp()` with `WidgetsFlutterBinding.ensureInitialized()` before `runApp()`. A try-catch wrapper was added so that if Firebase fails to initialize (e.g., during development without credentials), a helpful setup screen is shown instead of crashing.

Error 2 — Email Verification Not Enforced After Sign-In

Logic Bug — Unverified Users Accessing the App

```
Users who had not verified their email address were being routed directl
y to the main directory screen after sign-in.
```

The initial implementation used only `authStateChanges()` to determine if a user was authenticated. Since Firebase considers a user "signed in" regardless of email verification status, unverified users passed the auth gate.

Resolution

The `isAuthenticated` getter in `AuthProvider` was updated to require both an authenticated status *and* a verified email:

```
bool get isAuthenticated =>
    _status == AuthStatus.authenticated &&
    (_firebaseUser?.emailVerified ?? false);
```

Inside `_onAuthStateChanged`, `user.reload()` is called before reading `emailVerified` to ensure the local Firebase SDK reflects the latest server-side state. An `emailNotVerified` flag drives a banner in the auth screen that offers both "Resend email" and "I've verified ✓" actions, the latter calling `checkEmailVerified()` to reload and redirect without requiring the user to log out and back in.

1.2 Cloud Firestore Integration

Error 3 — Firestore Permission Denied on Read/Write

FirebaseException: permission-denied

```
[cloud_firestore/permission-denied] The caller does not have permission
to execute the specified operation.
```

All Firestore reads and writes failed immediately after the database was created. The default security rules created by Firebase were set to deny all access after the initial 30-day test window rule was not updated for production use.

Resolution

Firestore security rules were updated to allow authenticated users to read all listings, while restricting writes to the document owner:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Users can read/write their own profile
    match /users/{uid} {
      allow read, write: if request.auth != null && request.auth.uid == u
    }

    // All authenticated users can read listings
    // Only the creator can update or delete their own listing
    match /listings/{listingId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null;
      allow update, delete: if request.auth != null
        && request.auth.uid == resource.data.createdBy;
    }
  }
}
```

Error 4 — Real-Time Updates Not Rebuilding the UI

Logic Bug — Listings added in Firestore not appearing in the app

```
After adding a new listing via the form, the Directory screen did not up
date until the app was restarted.
```

The initial implementation used a `Future`-based one-time fetch (`get()`) rather than a real-time stream (`snapshots()`). Changes in Firestore were not pushed to the app automatically.

Resolution

`FirestoreService` was refactored to expose Dart `Stream`s using Firestore's `.snapshots()` listener. `ListingsProvider` subscribes to these streams via `StreamSubscription` and calls `notifyListeners()` on each emission, automatically rebuilding any `Consumer<ListingsProvider>` widgets in the tree:

```
Stream<List<ListingModel>> get listingsStream {
  return _db
      .collection('listings')
      .orderBy('timestamp', descending: true)
      .snapshots()
      .map((snapshot) => snapshot.docs
          .map((doc) => ListingModel.fromFirestore(doc))
          .toList());
}
```

1.3 Google Maps Integration

Error 5 — Blank/Grey Map on Android (Missing API Key)

Google Maps renders as a grey blank tile

```
W/GoogleMapActivity: Failed to load map. Error contacting Google server
s. This is probably an authentication issue (but could be due to network
errors).
```

The `GoogleMap` widget rendered as a solid grey tile on the Android emulator. No markers or map tiles were displayed. The logcat showed an authentication failure when contacting the Maps tile servers.

Resolution

A Google Maps API key was generated in Google Cloud Console with the **Maps SDK for Android** and **Maps SDK for iOS** APIs enabled. The key was added to `AndroidManifest.xml`:

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="YOUR_API_KEY"/>
```

For iOS, `GMSServices.provideAPIKey("YOUR_API_KEY")` was added to `AppDelegate.swift` before `super.application(...)`.

1.4 State Management with Provider

Early in development, Firestore queries were placed directly inside widget `build()` methods using `FutureBuilder`. This led to re-fetching data on every rebuild and made it impossible to share state across screens (e.g., the Directory and Map View showing different data).

The architecture was refactored to the three-layer pattern described in the Design Summary: every Firestore interaction goes through a `Service` class, is managed by a `Provider`, and is consumed by widgets via `Consumer<T>` or `context.watch<T>()`. This single change made all four main screens react instantly to the same underlying data stream.

2. GitHub Repository

Repository URL:
`https://github.com/Jongkuch1/kigali_services_directory`

The repository contains the complete Flutter project source code with Firebase integration. Below is a summary of the repository structure and commit history highlights.

Repository Structure

```
lib/
├── main.dart                    # App entry point, Provider setup, root gate
├── app_colors.dart              # Centralised design tokens (colours)
├── firebase_options.dart        # Auto-generated by FlutterFire CLI
│
├── models/
│   ├── listing_model.dart       # ListingModel with Firestore serialisation
│   └── user_profile_model.dart  # UserProfileModel
│
├── services/
│   ├── auth_service.dart        # Firebase Auth wrapper
│   └── firestore_service.dart   # Firestore CRUD + stream methods
│
├── providers/
│   ├── auth_provider.dart       # Auth state, email verification logic
│   ├── listings_provider.dart   # Listings CRUD, search/filter state
│   └── bookmarks_provider.dart  # Bookmarks state
│
├── screens/
│   ├── auth_screen.dart         # Login / Sign Up UI
│   ├── main_screen.dart         # BottomNavigationBar shell
│   ├── directory_screen.dart    # Browse listings, search, category filter
│   ├── my_listings_screen.dart  # User's own listings with Edit/Delete
│   ├── map_view_screen.dart     # Google Map with all listing markers
│   ├── settings_screen.dart     # User profile + notification toggle
│   ├── detail_screen.dart       # Listing detail, embedded map, directions
│   ├── add_edit_listing_screen.dart  # Create / Edit listing form
│   ├── bookmarks_screen.dart    # Saved listings
│   └── reviews_screen.dart      # Browse and write reviews
│
└── widgets/
    ├── listing_card.dart        # Reusable listing card widget
    ├── category_badge.dart      # Animated category filter chip
    └── star_rating.dart         # Half-star capable rating display
```

Key Commit Milestones

1. Initial Flutter project setup and dependency configuration
2. Firebase project created; FlutterFire CLI configuration applied
3. Firebase Authentication implemented (sign-up, login, email verification)
4. User profile Firestore document creation on registration
5. Firestore service layer and ListingModel created
6. ListingsProvider with real-time stream subscriptions

7. Directory screen with category sections and search bar
8. Add/Edit listing form with full validation
9. My Listings screen with Edit/Delete and confirmation dialogs
10. Detail screen with embedded Google Map and Get Directions button
11. Map View screen with all-listings markers
12. Settings screen with profile display and notification toggles
13. Bookmarks and Reviews features
14. Bug fixes: email verification UX, star rating half-star support

3. Design Summary

3.1 Firestore Database Structure

The application uses two top-level Firestore collections: `users` and `listings`. A third subcollection `reviews` is nested under each listing document.

> **Collections:** `users` · `listings` · `listings/{id}/reviews`

Collection: `users`

Each document is keyed by the Firebase Authentication UID and is created automatically when a user registers. This links every listing to its creator via the shared UID.

Field	Type	Description
`uid`	String	Firebase Auth UID (same as document ID)
`name`	String	User's display name entered at registration
`email`	String	Email address used for authentication
`createdAt`	Timestamp	Account creation timestamp

Collection: `listings`

Documents are auto-ID'd by Firestore. The `createdBy` field stores the UID of the authenticated user who created the listing, enabling user-scoped queries and Firestore security rule enforcement.

Field	Type	Description
`name`	String	Place or service name
`category`	String	One of: Cafés, Hospitals, Police, Libraries, Parks, Restaurants, Tourist
`address`	String	Physical street address in Kigali
`contact`	String	Phone number or contact info
`description`	String	Free-text description of the place or service
`lat`	Number	Geographic latitude (decimal degrees)
`lng`	Number	Geographic longitude (decimal degrees)
`rating`	Number	Rating from 0.0 to 5.0
`createdBy`	String	UID of the authenticated user who created the listing
`timestamp`	Timestamp	Creation date/time; used for ordering

Subcollection: ***`listings/{listingId}/reviews`***

Field	Type	Description
`author`	String	Display name of the reviewer
`text`	String	Review body text
`timestamp`	Timestamp	Time the review was posted

3.2 Listing Data Model

The `ListingModel` class in `lib/models/listing_model.dart` is the single source of truth for listing data throughout the application. It implements two serialisation methods to convert cleanly between Dart objects and Firestore documents:

```
class ListingModel {
  final String? id;
  final String name;
  final String category;
  final String address;
  final String contact;
  final String description;
  final double lat;
  final double lng;
  final double rating;
  final String createdBy;
  final DateTime timestamp;
  final List<ReviewModel> reviews;

  // Deserialise from Firestore DocumentSnapshot
  factory ListingModel.fromFirestore(DocumentSnapshot doc) { ... }

  // Serialise to Firestore Map
  Map<String, dynamic> toFirestore() { ... }
}
```

The `id` field is nullable because it is assigned by Firestore on creation and not known until the document is saved. The `reviews` list is fetched as part of the listing stream by querying the `reviews` subcollection in `FirestoreService`.

3.3 State Management Architecture

The application uses the **Provider** package for state management, organised in a strict three-layer architecture:

> **Data Flow:**
> Firestore / Firebase Auth → **Service Layer** → **Provider Layer** → **UI Layer**

Service Layer

`AuthService` and `FirestoreService` are plain Dart classes with no dependency on Flutter or Provider. They contain all Firebase API calls. This makes the business logic independently testable and keeps Firebase imports out of the UI files.

Provider Layer

Three `ChangeNotifier` providers manage application state:

- **AuthProvider** — listens to the Firebase auth state stream, enforces email verification, manages loading/error states for sign-in/sign-up, and exposes the current `UserProfileModel`.
- **ListingsProvider** — holds two `StreamSubscription`s (all listings and the authenticated user's own listings), exposes computed `filteredListings` for search and category filtering, and wraps CRUD operations in try/catch with error state exposure.
- **BookmarksProvider** — manages the user's bookmarked listing IDs via a Firestore subcollection with real-time sync.

UI Layer

Screens and widgets read state exclusively through `Consumer<T>` or `context.watch<T>()`. Write operations (CRUD, sign-in) are dispatched via `context.read<T>()` to avoid unnecessary rebuilds. No screen file imports `firebase_core`, `firebase_auth`, or `cloud_firestore` directly.

```
// Example: Directory screen reads filtered listings reactively
Consumer<ListingsProvider>(
  builder: (context, provider, _) {
    final listings = provider.filteredListings;
    // UI rebuilds automatically when Firestore data changes
    return ListView.builder( ... );
  },
)
```

3.4 Design Trade-offs & Technical Challenges

Trade-off 1 — Manual Coordinate Entry vs. Geocoding

The listing creation form requires latitude and longitude to be entered manually as decimal numbers. Integrating the Google Places Autocomplete API or a geocoding service would have provided a better user experience (type an address and get coordinates automatically). This was intentionally deferred because it requires an additional billed API (Places API) beyond the basic Maps SDK, and the manual approach is sufficient for demonstrating the core map integration requirement.

Trade-off 2 — Notification Simulation vs. Push Notifications

The Settings screen includes a Location-Based Notifications toggle that is persisted using `SharedPreferences`. Full push notification delivery would require Firebase Cloud Messaging (FCM) and a Cloud Functions backend to trigger location-based alerts. Given that the assignment explicitly permits a local simulation, `SharedPreferences` stores the user preference and the toggle state is reflected immediately in the UI without a backend.

Trade-off 3 — IndexedStack vs. Navigator for Bottom Navigation

The four main screens are wrapped in an `IndexedStack` rather than being pushed onto the Navigator stack. This means all four screens are instantiated when the app launches and remain alive throughout the session. The advantage is instant tab switching with no rebuild cost; the disadvantage is slightly higher memory usage. For this application's scale, the user experience benefit outweighs the memory cost. The Firestore stream subscriptions are started once in `MainScreen.initState()` and cancelled in `dispose()`, avoiding duplicate listeners.

Trade-off 4 — Flat Collection vs. Sub-collections for Listings

All listings are stored in a single flat `listings` collection rather than nested under each user. This makes it straightforward to query the full shared directory (all listings, ordered by timestamp) and separately query by `createdBy == uid` for "My Listings". If the collection were nested under `users/{uid}/listings`, a global directory query across all users would require Firestore Collection Group queries, which add complexity. The flat design is simpler and maps cleanly to the assignment requirements.